Experiment No. 2

AIM- To interface a push button as digital input and demonstrate LED control by toggling its state on each valid button press.

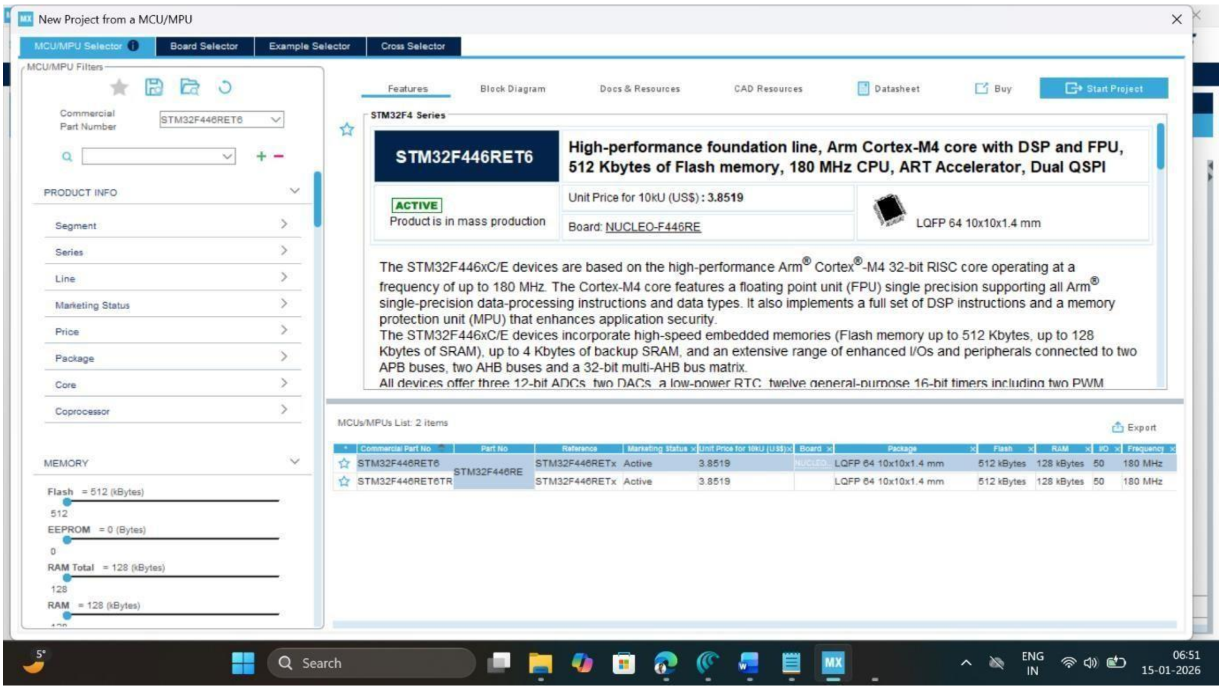
APPARATUS- STM32 Nucleo-F446RE development board, USB Type-A to Mini-B cable, STM32 CUBE IDE, STM32 CubeMX.

THEORY

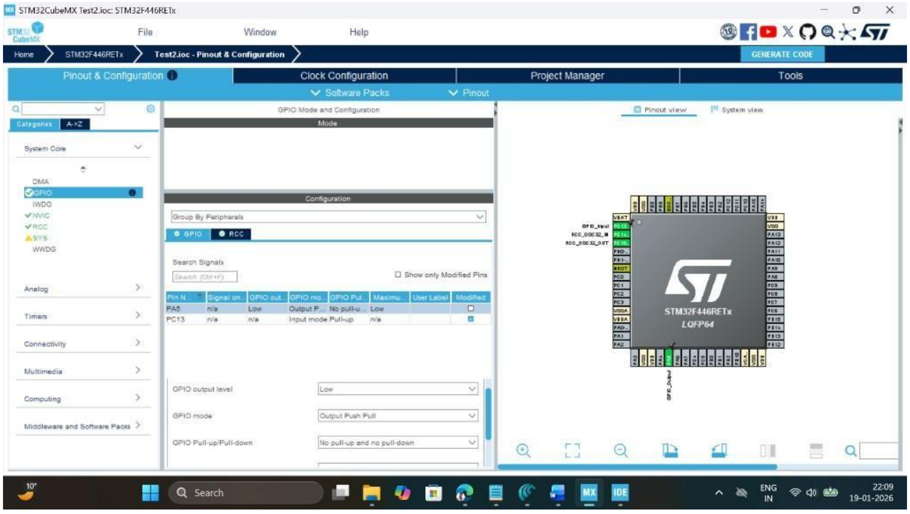
STM32F446RE provides multiple general-purpose I/O (GPIO) ports (GPIOA–GPIOH) that can be configured as input, output, alternate function, or analog through GPIO control registers or HAL. On the Nucleo-F446RE board, the green user LED **LD2** is connected to Arduino D13, which corresponds to microcontroller pin **PA5** on most board revisions. When PA5 is configured as a **push-pull output**, the pin can be driven to logic high (near 3.3 V) or logic low (0 V), which in turn turns the LED on and off through an on-board current-limiting. Using STM32CubeIDE and HAL, functions like HAL_GPIO_TogglePin() and HAL_Delay() can be used inside an infinite loop to generate a periodic LED blink without directly programming registers.

PROCEDURE

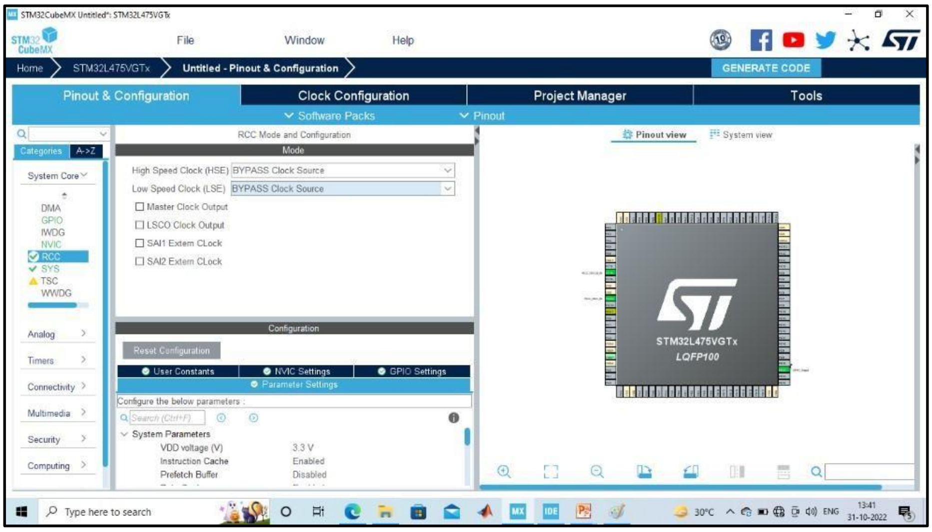
1. Open CubeMX and Click on ACCESS TO MCU SELECTOR.



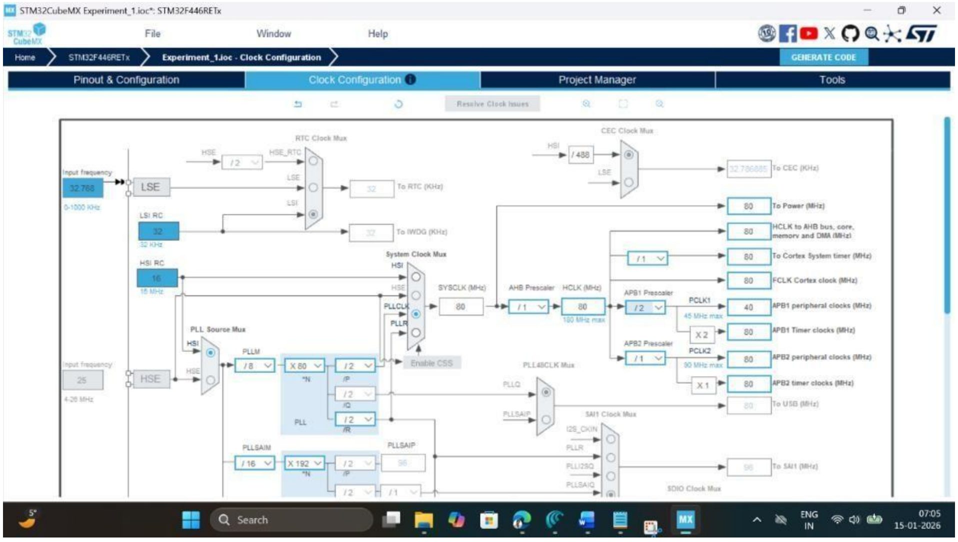
2. Write and select the microcontroller as you want to use and then click start the project. Select commercial part number STM32L446RE.
3. In system Core select GPIO and right click on pin PA5 to make it as GPIO output pin as shown below as LED (Inbuilt LED) and PC13 as Pushbutton (USER) relates to it in the board.



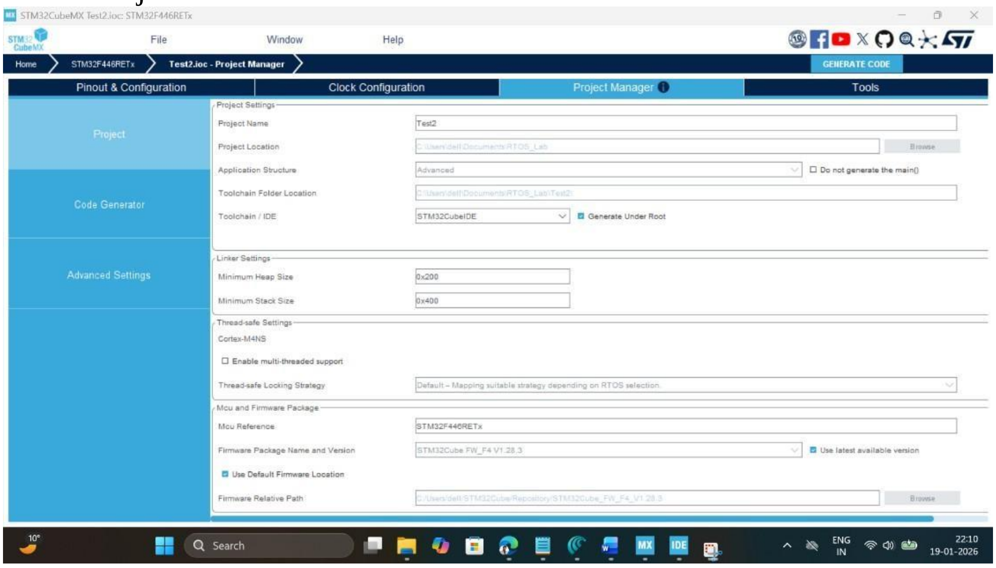
4. **Clock Selection**: Select the internal clock by clicking RCC and selecting BYPASS clock source



5. **Clock Configuration:** Configure clock as shown below to get maximum internal clock frequency.



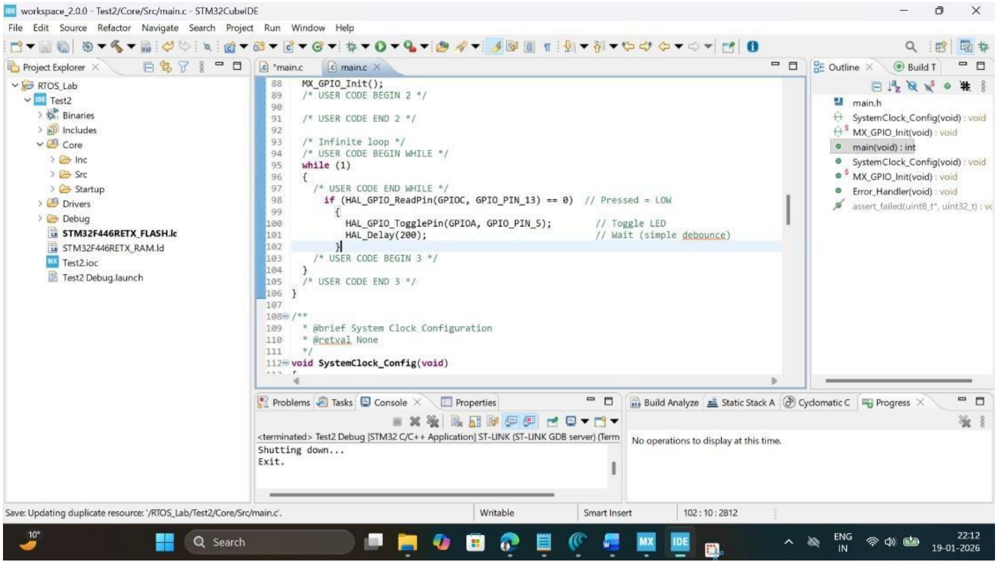
6. Write Project name and select IDE.



7. Click on generate code.
8. Click on open project.
9. Click ‘OK’ on the generated popup showing “successfully imported the project on the workspace” and the project will be successfully added to the cube IDE.
10. Open the main source code on Cube IDE by clicking on **main.c**
11. To generate bin file and hex file, right click on project and click properties.
12. Expand C/C++ Build, click on setting, click on MCU post build outputs and select convert to binary file as well as convert to hex file and click “Apply and Close ".
13. After configuration write only below two instructions in the user while loop and compile the program and ensure there will be zero error after compiling.

```
while (1)
{
 // Read onboard button (PC13)  if
(HAL_GPIO_ReadPin(GPIOC, GPIO_+
PIN_13) == 0)  // Pressed = LOW
 {
  HAL_GPIO_TogglePin(GPIOA,  GPIO_PIN_5);        // Toggle LED
```

```
    HAL_Delay(200);                    // Wait (simple debounce)
  }
}
```



14. To run this program, connect the board with USB cable, click on build, click ‘ok’ and click ‘switch’ on popups.
15. Now you are in debugging mode, click ‘Resume’ to execute the program.

OBSERVATION:

Button Press vs LED State Observation

Trial No.	**Button Action**	**Button Pin State (PC13)**	**LED Pin State (PA5) Before**	**LED Pin State (PA5) After**	**Observation**
1	Initial (No press)	High	Low	**Low**	**LED is Off**
1	1st Press	Low	Low	High	LED is on
2	Release	High	High	High	LED is on
3	2nd Press	Low	High	Low	LED is Off
4	3rd Press	Low	Low	High	LED is on
5	Rapid Press	Toggle	Dependent on toggling	toggles	flickering

RESULT

The push button connected to pin **PC13** was successfully interfaced as a digital input, and pin **PA5** was configured as a digital output to control the on-board LED **LD2** on the STM32F446RE Nucleo board. On each valid button press, the LED toggled between ON and OFF states as programmed. The LED maintained its state after the button was released. The behavior observed during the experiment matched the values recorded in the observation table, confirming correct GPIO configuration and proper functioning of the HAL GPIO read and toggle operations.

Reflection Summary

From this experiment, it was understood that the microcontroller detects a button press by continuously reading the digital logic level of the input pin rather than recognizing a single physical press. The experiment demonstrated how a change in logic state from HIGH to LOW can be used to control an output device such as an LED.

This experiment also showed the effect of switch bouncing, where a single button press may be detected multiple times. The use of a small delay in the program helped reduce this effect. Overall, the experiment highlighted the importance of correct GPIO configuration and basic timing control for reliable input detection and output control. The concepts learned from this experiment are useful for future embedded system applications involving digital inputs and outputs.